Trabajo Práctico Integrador – Programación II

Food Store – Sistema de Gestión de Pedidos de Comida (Consola)

Se propone desarrollar un sistema de consola llamado **Food Store**, orientado a la gestión de un negocio de comidas, aplicando el paradigma de Programación Orientada a Objetos (POO) e implementado en Python.

El sistema permitirá gestionar categorías, productos, usuarios y pedidos según el diagrama UML provisto, realizando operaciones de alta, baja, modificación y consulta desde un menú de opciones en consola.

El sistema inicia con un **inicio de sesión básico**: según el rol del usuario que ingresa, se muestra el menú de administrador (alta, baja, modificación y consulta) o el menú de cliente (solo consulta).

Toda la información vive en memoria mientras el programa se ejecuta, guardada en colecciones. Al cerrar el programa, los datos se pierden; esto es esperado.

El desarrollo se guía por un backlog de épicas e historias de usuario (ver Anexo): cada historia se transforma en una opción concreta del menú. Además, se deben incorporar validaciones de entrada, excepciones propias y una separación clara de responsabilidades entre modelo, servicios y consola.

Contenido

Alcance 2
Información del proyecto 2
Objetivos de aprendizaje 2
Modelo (UML) y entidades 3
Inicio de sesión y roles 5
Interfaz de consola (menú) 6
Arquitectura recomendada 7
Reglas mínimas de negocio 8
Definición de completado 9
Criterios de evaluación (orientativos) 9
Entregables 9
Anexo – Historias de usuario 11
Épica 0 – Acceso al sistema 11
Épica 1 – Gestión de categorías 11
Épica 2 – Gestión de productos 12
Épica 3 – Gestión de usuarios 13
Épica 4 – Gestión de pedidos y detalles 13

Alcance

- Toda la interacción ocurre en consola mediante menús.
- Se aplica POO según el UML: encapsulamiento, constructores, herencia, polimorfismo y sobrescritura.
- Se implementa un **inicio de sesión** con mail y contraseña, y el menú cambia según el rol: **ADMIN** gestiona todo, **USUARIO** solo consulta.
- Se usan **colecciones** para almacenar y gestionar los objetos en memoria.
- Se implementa una **interfaz** para definir un comportamiento común (`Calculable`).
- Se manejan **excepciones**, incluidas excepciones propias, para validar errores y evitar cierres inesperados.
- Se escriben **type hints** en todos los atributos, parámetros y valores de retorno.

Información del proyecto

Aspecto	Definición
Modalidad	Aplicación de consola con menús
Lenguaje	Python 3.12 o superior
Almacenamiento	En memoria (colecciones)
Capas	Entidades (modelo) + Servicios (lógica) + Consola (menús)
Bibliotecas	Solo biblioteca estándar (`abc`, `enum`, `datetime`)

Objetivos de aprendizaje

Al finalizar el TPI, el proyecto debe evidenciar que el estudiante puede:

1. Modelar correctamente clases y relaciones del UML: herencia, asociación 1..N y N..1, agregación y composición.
2. Aplicar herencia desde una clase abstracta y sobrescribir métodos (`__str__`, `__eq__`).
3. Proteger el estado de los objetos con encapsulamiento y validaciones en los setters.
4. Usar colecciones para gestionar las entidades.
5. Implementar una interfaz para estandarizar un comportamiento (`Calculable`).
6. Crear, lanzar y capturar excepciones propias (por ejemplo `EntidadNoEncontradaError`, `StockInsuficienteError`).
7. Aplicar polimorfismo para que cada rol tenga su propio menú, sin condicionales por rol repartidos por el código.
8. Documentar el contrato de cada clase y método con type hints.

Modelo (UML) y entidades

El sistema se basa en el siguiente diseño orientado a objetos: varias clases comparten atributos comunes heredando de una clase abstracta, y `Pedido` implementa una interfaz. Los nombres de atributos y métodos se escriben en `snake_case`; los tipos indicados son los type hints que deben aparecer en el código.

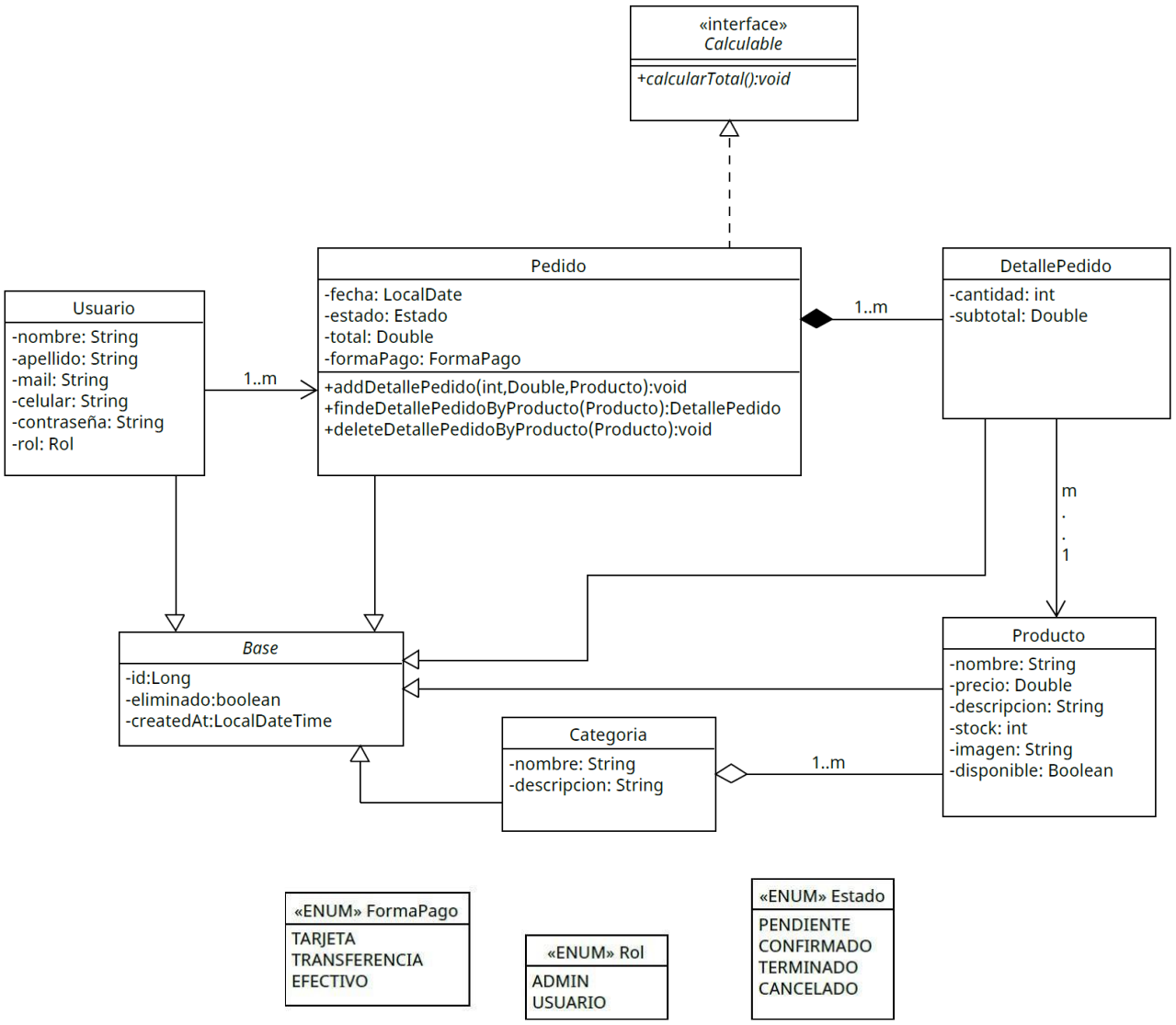


Figura 1. Diagrama de clases de Food Store.

Clase	Hereda / implementa	Atributos propios (con tipo)	Relaciones
`Base` (abstracta)	`ABC`	`id: int`, `eliminado: bool`, `creado_en: datetime`	Superclase de todas las entidades
`Categoria`	`Base`	`nombre: str`, `descripcion: str`	Agregación 1..N con `Producto`
`Producto`	`Base`	`nombre: str`, `precio: float`, `descripcion: str`, `stock: int`, `imagen: str`, `disponible: bool`	N..1 con `Categoria`

Clase	Hereda / implementa	Atributos propios (con tipo)	Relaciones
`Usuario`	`Base`	`nombre: str`, `apellido: str`, `mail: str` (único), `celular: str`, `contrasenia: str`, `rol: Rol`	Asociación 1..N con `Pedido`
`Pedido`	`Base`, `Calculable`	`fecha: date`, `estado: Estado`, `total: float`, `forma_pago: FormaPago`	N..1 con `Usuario`; composición 1..N con `DetallePedido`
`DetallePedido`	`Base`	`cantidad: int`, `subtotal: float`	N..1 con `Producto`; vive dentro de un `Pedido`

Enumerados (enum.Enum)

- `Rol`: ADMIN, USUARIO
- `Estado`: PENDIENTE, CONFIRMADO, TERMINADO, CANCELADO
- `FormaPago`: TARJETA, TRANSFERENCIA, EFECTIVO

Interfaz Calculable

Define un único método abstracto `calcular_total() -> None`, que actualiza el total del objeto que la implementa. No tiene atributos.

Métodos propios de Pedido (obligatorios, con estas firmas)

```
def agregar_detalle(self, cantidad: int, producto: Producto) -> None: ...
def buscar_detalle_por_producto(self, producto: Producto) -> DetallePedido | None: ...
def eliminar_detalle_por_producto(self, producto: Producto) -> None: ...
def calcular_total(self) -> None: ...
```

El precio del detalle se toma del propio producto en el momento de agregarlo, para que el subtotal no dependa de un dato que escribe el operador.

Requisitos de implementación del modelo

- `Base` no se puede instanciar y asigna el `id` automáticamente con un contador de clase compartido por todas las entidades (o uno por clase, a elección, justificado en el informe).
- `creado_en` se asigna en el constructor con la fecha y hora actuales; no se pide al usuario.
- Todos los atributos son privados y se exponen con `@property`. Solo tienen setter los atributos que el sistema permite modificar, y el setter valida.
- `contrasenia` es la excepción: no tiene property de lectura. `Usuario` ofrece `verificar_contrasenia(contrasenia: str) -> bool`, así la contraseña nunca sale del objeto.
- Cada entidad sobrescribe `__str__` con una línea útil para los listados de consola.
- `__eq__` compara por `id` en todas las entidades (se puede definir una sola vez en `Base`).
- `Pedido` es el único que crea objetos `DetallePedido`: el detalle nace y muere con su pedido.
- La lista de detalles de un pedido no se expone para ser modificada desde afuera (la property devuelve una copia o una tupla).

Ejemplo del estilo esperado (solo la estructura; no es la solución)

```
from abc import ABC, abstractmethod


class Calculable(ABC):
    @abstractmethod
    def calcular_total(self) -> None:
        ...


class Producto(Base):
    def __init__(self, nombre: str, precio: float, stock: int, categoria: 'Categoria') ->
None:
        super().__init__()
        self.nombre = nombre
        self.precio = precio
        ...

    @property
    def precio(self) -> float:
        return self.__precio

    @precio.setter
    def precio(self, valor: float) -> None:
        if valor < 0:
            raise PrecioInvalidoError('El precio no puede ser negativo')
        self.__precio = valor
```

Inicio de sesión y roles

Al ejecutar el programa se piden mail y contraseña. Si coinciden con un usuario activo, se abre el menú que corresponde a su rol; si no, se informa y se vuelve a pedir.

```
=== FOOD STORE – INICIO DE SESIÓN ===
Mail: admin@foodstore.com
Contraseña: ********

Bienvenido/a, Ana Pérez [ADMIN]
```

Reglas del inicio de sesión

- La verificación la hace `UsuarioServicio.autenticar(mail: str, contrasenia: str) -> Usuario`, que usa `verificar_contrasenia(...)` y lanza `CredencialesInvalidasError` si el mail no existe, el usuario está eliminado o la contraseña no coincide.
- El mensaje es el mismo en los tres casos ("Mail o contraseña incorrectos"), para no revelar qué dato falló.
- Hay hasta 3 intentos; al tercer error el programa informa y termina.
- `main()` carga un **administrador inicial** (por ejemplo `admin@foodstore.com` / `admin123`), porque sin él nadie podría ingresar. También puede cargar datos de ejemplo.
- El usuario que inició sesión (la sesión) se guarda en una variable local de `main()` y se pasa al menú; no es una variable global.

- "Cerrar sesión" vuelve a la pantalla de inicio de sesión sin perder los datos cargados; "Salir" termina el programa.
- Opcional: ocultar la contraseña al escribirla con `getpass.getpass()`.

Permisos por rol

Sección	ADMIN	USUARIO (cliente)
Categorías	Listar, crear, editar, eliminar	Listar
Productos	Listar, crear, editar, eliminar	Listar solo los disponibles y con stock
Usuarios	Listar, crear, editar, eliminar	Ver sus propios datos
Pedidos	Listar todos, crear, cambiar estado y forma de pago, eliminar	Ver solo sus propios pedidos y sus detalles

Menú según el rol (polimorfismo)

El menú no se elige con `if` por rol repartidos por la consola. Se modela una clase abstracta `Menu` con un método abstracto `mostrar()` y una subclase por rol: `MenuAdministrador` y `MenuCliente`. El rol se pregunta en un único lugar, al crear el menú; después el programa solo llama a `menu.mostrar()` sin saber qué menú es.

```
class Menu(ABC):
    def __init__(self, usuario: Usuario) -> None:
        self._usuario = usuario

    @abstractmethod
    def mostrar(self) -> bool:
        ...  # True = cerrar sesión, False = salir del programa


def crear_menu(usuario: Usuario, ...) -> Menu:
    if usuario.rol == Rol.ADMIN:
        return MenuAdministrador(usuario, ...)
    return MenuCliente(usuario, ...)
```

Cada subclase recibe además los servicios que necesita (los `...` del ejemplo). Agregar un rol nuevo significa agregar una subclase, sin tocar los menús existentes.

Interfaz de consola (menú)

Después de iniciar sesión se muestra el menú del rol. El texto puede variar, la idea es la misma. Menú de administrador:

```
=== FOOD STORE – ADMINISTRADOR (Ana Pérez) ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
9. Cerrar sesión
0. Salir
Seleccione:
```

Menú de cliente (solo consulta):

```
=== FOOD STORE – CLIENTE (Juan Gómez) ===
1. Ver categorías
2. Ver productos disponibles
3. Mis pedidos
4. Mis datos
9. Cerrar sesión
0. Salir
Seleccione:
```

Cada opción del administrador abre un submenú con las operaciones de alta, baja, modificación y consulta. Ejemplo para Categorías:

```
--- CATEGORÍAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione:
```

Pautas del menú

- Valida todas las entradas: opciones fuera de rango, ids inexistentes, números mal ingresados, textos vacíos.
- Un error nunca cierra el programa: se informa y se vuelve al menú.
- Cada operación muestra un mensaje claro de éxito o de error.
- Antes de pedir un id se pueden listar los elementos disponibles.
- Las opciones de enumerados (rol, estado, forma de pago) se eligen por número, nunca escribiendo el texto.
- Al editar, dejar el campo vacío (Enter) conserva el valor anterior.
- El menú de cliente no ofrece ninguna opción que modifique datos.
- No se usan variables globales: los servicios y la sesión se crean en `main()` y se pasan a los menús.

Arquitectura recomendada

El proyecto se organiza en tres capas, cada una en su paquete, y la dependencia va siempre en un solo sentido: consola → servicios → entidades.

```
food_store/
├── main.py              # crea los servicios y arranca el menú principal
├── pruebas.py           # pruebas del modelo y servicios, sin menú
├── entidades/           # clases del UML (Base, Categoria, Producto, ...)
│   └── calculable.py    # interfaz Calculable
├── enums/               # Rol, Estado, FormaPago
├── excepciones/         # FoodStoreError y sus subclases
├── servicios/           # un servicio por entidad (CategoriaServicio, ...)
└── consola/             # Menu, MenuAdministrador, MenuCliente y lectura validada
```

Responsabilidad de cada capa

Capa	Hace	No hace
Entidades	Guardan su estado, lo validan en los setters, se muestran con `__str__`	Usar `input()` o `print()`; conocer las colecciones
Servicios	Tienen la colección de su entidad; buscan por id, aplican reglas de negocio que involucran a varios objetos (mail único, nombre de categoría repetido)	Leer del teclado o imprimir
Consola	Pide datos, arma los objetos a través de los servicios, captura excepciones y muestra mensajes	Contener reglas de negocio

Excepciones propias sugeridas (todas heredan de FoodStoreError)

- `EntidadNoEncontradaError`: id inexistente o eliminado.
- `DatoInvalidoError`: texto vacío, formato incorrecto. Puede tener subclases como `PrecioInvalidoError`.
- `StockInsuficienteError`: cantidad pedida mayor al stock.
- `DuplicadoError`: mail o nombre de categoría repetido.
- `OperacionNoPermitidaError`: transición de estado inválida, categoría con productos activos, administrador que intenta eliminarse.
- `CredencialesInvalidasError`: mail o contraseña incorrectos, o usuario eliminado.

La consola captura `FoodStoreError` para mostrar mensajes de negocio y `ValueError` para conversiones fallidas de `int()` / `float()`.

Reglas mínimas de negocio

Además del alta, baja, modificación y consulta, el sistema debe respetar estas reglas para que el dominio tenga sentido:

- No se puede crear ni editar un producto con precio < 0 o stock < 0: se lanza una excepción.
- No se puede crear un pedido sin usuario, ni con un usuario eliminado.
- La cantidad de un detalle debe ser > 0 y no puede superar el stock del producto.
- Un pedido no puede tener dos detalles del mismo producto: si se agrega de nuevo, se suma la cantidad al detalle existente.
- El mail de usuario es único (se valida recorriendo la colección).
- El nombre de categoría es único, sin distinguir mayúsculas.
- No se puede eliminar una categoría con productos activos.
- Transiciones de estado válidas: PENDIENTE → CONFIRMADO → TERMINADO; PENDIENTE o CONFIRMADO → CANCELADO. TERMINADO y CANCELADO no cambian más.
- Al cancelar o eliminar un pedido no terminado, el stock de sus productos se devuelve.
- Toda baja es lógica: `eliminado = True`, el objeto sigue en la colección.
- Solo un usuario con rol ADMIN puede crear, editar o eliminar datos.
- Un usuario eliminado no puede iniciar sesión.
- Un administrador no puede eliminarse a sí mismo mientras tiene la sesión iniciada.

Definición de completado

- ☐ El programa corre sin errores con `python main.py`.
- ☐ Se puede hacer alta, baja, modificación y consulta de categorías, productos, usuarios y pedidos desde la consola.
- ☐ `main.py` no contiene reglas de negocio.
- ☐ Se puede iniciar sesión como ADMIN y como USUARIO, y cada uno ve solo las opciones de su rol.
- ☐ Ningún dato mal ingresado cierra el programa (salvo al agotar los 3 intentos de inicio de sesión).
- ☐ Todos los atributos, parámetros y retornos tienen type hints.
- ☐ `pruebas.py` corre y muestra en pantalla los casos válidos y los errores esperados.

Criterios de evaluación (orientativos)

Criterio	Qué se mira
POO	Modelado fiel al UML; clase abstracta e interfaz bien usadas; encapsulamiento con properties; composición real en `Pedido`; menú por rol resuelto con polimorfismo; `__str__` útil
Calidad de código	Nombres claros, separación por capas, funciones cortas, type hints completos y correctos
Robustez	Validaciones, excepciones propias, permisos por rol respetados, mensajes claros, menú que nunca se cae
Documentación y video	Informe completo y claro; video que recorre el flujo completo

Entregables

1. Repositorio GitHub

- Proyecto Python completo, organizado por paquetes.
- `README.md` en la raíz con: cómo ejecutar el proyecto (versión de Python y comando), credenciales del administrador inicial, enlace al video con permisos públicos de visualización y enlace al PDF del informe (o el PDF subido a la raíz).

2. Informe en PDF

El documento debe estar correctamente ordenado, con las hojas numeradas y formato profesional. Debe incluir:

- **Carátula**: institución, carrera, materia (Programación II), título del proyecto, datos del estudiante y fecha de entrega.
- **Índice** con números de página.
- **Marco teórico**: conceptos del paradigma aplicados (clase abstracta, interfaz, herencia, polimorfismo, encapsulamiento, composición/agregación, excepciones, colecciones) y una breve descripción de Python y de los type hints.
- **Decisiones técnicas y arquitectura**: cómo se abordó la solución, cómo se organizaron los paquetes y capturas de pantalla del sistema funcionando.
- **Dificultades y soluciones**: principales obstáculos y cómo se resolvieron.

- **Bibliografía / webgrafía**: documentación oficial y recursos usados.
- Enlaces al repositorio y al video.

3. Video demostración

Grabación de 10 a 15 minutos con el sistema funcionando. Debe recorrer el flujo completo, incluidos el inicio de sesión con ambos roles, al menos un error de validación y un pedido cancelado por falta de stock.

Método de entrega

La entrega se realiza **exclusivamente en un archivo .zip** que contenga el código fuente completo y el informe en PDF. No se consideran entregas incompletas o que no respeten el formato indicado.

Anexo – Historias de usuario

Este anexo es el backlog mínimo: cada historia se cumple implementando su opción del menú y la lógica que la sostiene.

Notas

- Se ingresa con inicio de sesión. **Administrador** = usuario con rol ADMIN; **cliente** = usuario con rol USUARIO.
- Todas las bajas son lógicas (`eliminado = True`).
- Las interacciones se realizan mediante menús de consola.

Épica 0 – Acceso al sistema

HU-ACC-01 – Iniciar sesión. Como usuario registrado, quiero ingresar con mi mail y contraseña para acceder a las opciones de mi rol.

Criterios de aceptación:

- Se piden mail y contraseña.
- Si son correctos y el usuario no está eliminado, se muestra un saludo con su nombre y rol, y el menú de su rol.
- Si no, se informa "Mail o contraseña incorrectos" sin indicar cuál dato falló.
- Al tercer intento fallido, el programa informa y termina.

HU-ACC-02 – Menú según el rol. Como responsable del negocio, quiero que cada rol vea solo sus opciones para que un cliente no pueda modificar datos.

Criterios de aceptación:

- ADMIN ve el menú completo de gestión.
- USUARIO ve solo opciones de consulta: categorías, productos disponibles, sus pedidos y sus datos.
- El menú se resuelve con polimorfismo: clase abstracta `Menu` y una subclase por rol.

HU-ACC-03 – Cerrar sesión. Como usuario, quiero cerrar sesión para que otra persona pueda ingresar.

Criterios de aceptación:

- La opción "Cerrar sesión" vuelve a la pantalla de inicio de sesión.
- Los datos cargados se mantienen en memoria.

Épica 1 – Gestión de categorías

HU-CAT-01 – Listar categorías. Como usuario (administrador o cliente), quiero listar las categorías para conocer el catálogo.

Criterios de aceptación:

- Se muestran solo las categorías no eliminadas, con id, nombre y descripción.
- Si no hay ninguna, se informa "No hay categorías cargadas".

HU-CAT-02 – Crear categoría. Como administrador, quiero crear una categoría para organizar el catálogo.

Criterios de aceptación:

- Se piden nombre y descripción, ninguno vacío.

- No se permite un nombre repetido (sin distinguir mayúsculas): se informa el error.
- Se agrega a la colección y se informa el id generado.

HU-CAT-03 – Editar categoría. Como administrador, quiero editar una categoría para mantenerla actualizada.

Criterios de aceptación:

- Se selecciona por id; si no existe o está eliminada, se informa y no se modifica nada.
- Se actualiza nombre y/o descripción (Enter conserva el valor) y se confirma.

HU-CAT-04 – Eliminar categoría (baja lógica). Como administrador, quiero dar de baja una categoría sin perder historial.

Criterios de aceptación:

- Se pide id y confirmación (S/N).
- Si tiene productos activos, no se elimina y se informa cuántos tiene.
- Si no, se marca `eliminado = True` y deja de aparecer en los listados.

Épica 2 – Gestión de productos

HU-PROD-01 – Listar productos. Como usuario (administrador o cliente), quiero listar productos para consultar sus datos y stock.

Criterios de aceptación:

- Se muestran solo los no eliminados, con id, nombre, precio, stock y nombre de la categoría.
- El cliente ve solo los productos disponibles y con stock mayor a 0.
- Opcional: listar filtrando por categoría.

HU-PROD-02 – Crear producto. Como administrador, quiero crear un producto asociado a una categoría.

Criterios de aceptación:

- Se piden nombre, descripción, precio, stock, imagen, disponibilidad (S/N) y categoría por id.
- Validaciones: nombre no vacío, precio >= 0, stock >= 0, categoría existente y no eliminada.
- Se agrega a la colección y se informa el id generado.

HU-PROD-03 – Editar producto. Como administrador, quiero actualizar precio, stock u otros datos de un producto.

Criterios de aceptación:

- Se selecciona por id; si no existe o está eliminado, se informa y se cancela.
- Se pueden modificar uno o más campos; Enter conserva el valor anterior.
- Se aplican las mismas validaciones que al crear.

HU-PROD-04 – Eliminar producto (baja lógica). Como administrador, quiero retirar un producto del catálogo sin borrar su historia.

Criterios de aceptación:

- Se pide id y confirmación; se marca `eliminado = True`.
- No aparece en listados ni puede agregarse a pedidos nuevos.
- Los pedidos existentes que lo incluyen siguen mostrándose correctamente.

Épica 3 – Gestión de usuarios

HU-USR-01 – Listar usuarios. Como administrador, quiero listar usuarios para consultar sus datos de contacto.

Criterios de aceptación:

- Se muestran solo los no eliminados, con id, nombre, apellido, mail y rol (nunca la contraseña).
- Si no hay usuarios, se informa.

HU-USR-02 – Crear usuario. Como administrador, quiero crear un usuario para asociarlo a pedidos.

Criterios de aceptación:

- Se piden nombre, apellido, mail, celular, contraseña y rol (por número).
- El mail no puede estar vacío y debe tener formato básico (contener "@" y ".").
- Si el mail ya existe, se informa y se pide otro.
- Se agrega a la colección y se informa el id generado.

HU-USR-03 – Editar usuario. Como administrador, quiero corregir los datos de un usuario.

Criterios de aceptación:

- Se selecciona por id; si no existe o está eliminado, se informa.
- Si se cambia el mail, se valida que siga siendo único.
- Se confirma la actualización.

HU-USR-04 – Eliminar usuario (baja lógica). Como administrador, quiero dar de baja un usuario para que no se use en pedidos nuevos.

Criterios de aceptación:

- Se pide id y confirmación; se marca `eliminado = True`.
- No aparece en listados, no puede elegirse al crear pedidos y no puede iniciar sesión.
- Un administrador no puede eliminarse a sí mismo: se informa y no cambia nada.
- Sus pedidos existentes se siguen pudiendo consultar.

HU-USR-05 – Ver mis datos. Como cliente, quiero ver mis datos para comprobar que están correctos.

Criterios de aceptación:

- Se muestran nombre, apellido, mail, celular y rol del usuario con sesión iniciada, nunca la contraseña.

Épica 4 – Gestión de pedidos y detalles

HU-PED-01 – Listar pedidos. Como administrador, quiero listar pedidos para consultar su estado y total.

Criterios de aceptación:

- Se muestran solo los no eliminados, con id, usuario, estado, forma de pago, total y fecha.
- Opcional: filtrar por usuario y ver los detalles de un pedido.

HU-PED-02 – Crear pedido con detalles. Como administrador, quiero registrar un pedido con sus productos y cantidades.

Criterios de aceptación:

- Se elige el usuario por id (existente y no eliminado) y la forma de pago por número. El pedido arranca en PENDIENTE con la fecha del día.
- Se agregan uno o más detalles usando **obligatoriamente** `agregar_detalle(...)` de `Pedido`.
- El subtotal de cada detalle es cantidad × precio del producto al momento de agregarlo.
- El total se establece **obligatoriamente** con `calcular_total()` de la interfaz `Calculable`.
- Si al agregar un detalle ocurre una excepción (por ejemplo, falta de stock), se informa y se cancela la creación: el pedido no se agrega a la colección y no se modifica ningún stock.
- Si todo es válido, el pedido se agrega a la colección, se descuenta el stock y se informa el id y el total.

HU-PED-03 – Actualizar estado y/o forma de pago. Como administrador, quiero reflejar el avance de un pedido.

Criterios de aceptación:

- Se selecciona por id; el estado y la forma de pago se eligen por número.
- Solo se permiten las transiciones de estado definidas en las reglas de negocio; si no es válida, se informa y no cambia nada.
- Si pasa a CANCELADO, se devuelve el stock.
- Se informa el éxito.

HU-PED-04 – Eliminar pedido (baja lógica). Como administrador, quiero ocultar un pedido sin perder la auditoría.

Criterios de aceptación:

- Se pide id y confirmación.
- Se marca `eliminado = True` en el pedido y en sus detalles.
- Si no estaba TERMINADO ni CANCELADO, se devuelve el stock.
- No aparece en los listados y las colecciones quedan consistentes.

HU-PED-05 – Consultar mis pedidos. Como cliente, quiero ver mis pedidos y su estado para hacer el seguimiento.

Criterios de aceptación:

- Se muestran solo los pedidos no eliminados del usuario con sesión iniciada, con id, estado, forma de pago, total y fecha.
- Se puede elegir uno por id y ver sus detalles (producto, cantidad, subtotal) y el total.
- Si el id corresponde a un pedido de otro usuario, se informa como inexistente.